

Length of a Loop in a Linked List

Personal DSA Notes

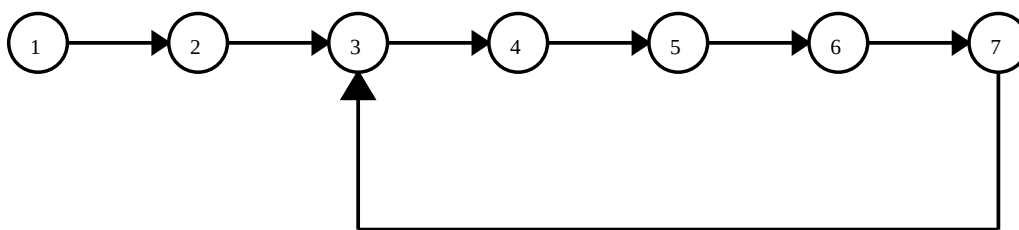
1. Introduction

The task is to find the number of nodes in a loop. If the linked list has no loop, return 0.

2. Problem Statement

Given the head of a linked list, return the length of the loop if one exists. A loop contains at least one node that can be reached again by following next pointers.

Loop length = 7 nodes

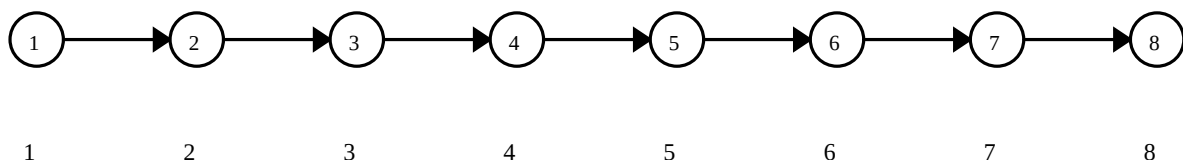


Starting at any loop node and returning to it counts 7 steps.

3. Approach 1 — Hashing with a Timer

During traversal, store each complete node reference together with the timer value at which that node was first visited. When the same node is encountered again, subtract its first-visit timer from the current timer. That difference is the loop length.

Important: store the node object/reference, not only its value, because different nodes may contain the same value.



Timer stored when each node is first visited.

When node 3 is seen again at timer 10: $10 - 3 = 7$.

$$10 - 3 = 7$$

Hashing Pseudocode
map = empty map

```

temp = head
timer = 1

while temp != null:
    if temp exists in map:
        return timer - map.get(temp)

    map.put(temp, timer)
    timer = timer + 1
    temp = temp.next

return 0

```

Java Implementation

```

class Solution {
    public int findLengthOfLoop(ListNode head) {
        Map<ListNode, Integer> map = new HashMap<>();

        ListNode temp = head;
        int timer = 1;

        while (temp != null) {
            if (map.containsKey(temp)) {
                return timer - map.get(temp);
            }

            map.put(temp, timer);
            timer++;
            temp = temp.next;
        }

        return 0;
    }
}

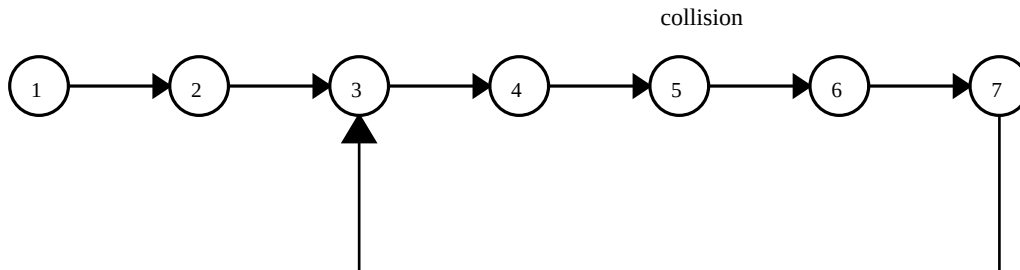
```

Time: $O(N)$ traversal plus map operations. Depending on the map implementation, lookup/insertion may be $O(1)$ expected or $O(\log N)$. Extra space: $O(N)$.

4. Approach 2 — Tortoise and Hare

First detect whether a loop exists using slow and fast pointers. Slow moves one step; fast moves two steps. If they collide, a loop exists. If fast reaches null, return 0.

Phase 1: slow = 1 step, fast = 2 steps



They collide inside the loop. Then count around the loop until returning to the same node.

```
slow = head
fast = head

while fast != null && fast.next != null:
    slow = slow.next
    fast = fast.next.next

    if slow == fast:
        return countLoop(slow)

return 0
```

5. Counting the Loop Length

Once slow and fast collide, the collision node is definitely inside the loop. Start a counter at 1, move one pointer one node at a time, and stop when it returns to the same collision node.

```
private int countLoop(ListNode node) {
    int count = 1;
    ListNode current = node.next;

    while (current != node) {
        count++;
        current = current.next;
    }

    return count;
}
```

6. Complete Java Code

```
class Solution {

    private int countLoop(ListNode node) {
        int count = 1;
        ListNode current = node.next;

        while (current != node) {
            count++;
            current = current.next;
        }

        return count;
    }
}
```

```

public int findLengthOfLoop(ListNode head) {
    ListNode slow = head;
    ListNode fast = head;

    // Phase 1: detect loop
    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;

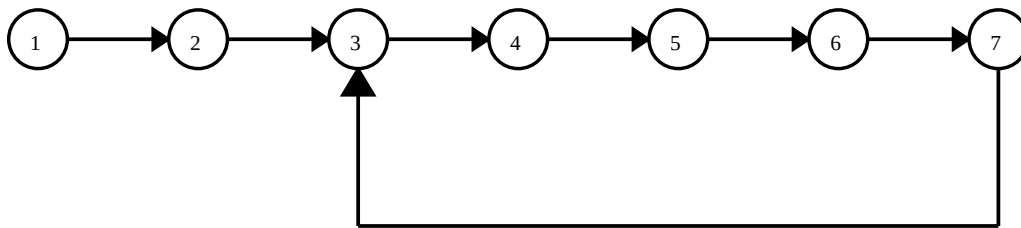
        if (slow == fast) {
            // Phase 2: count loop nodes
            return countLoop(slow);
        }
    }

    return 0;
}

```

7. Visual Representation of the Optimal Approach

Loop length = 7 nodes



Starting at any loop node and returning to it counts 7 steps.

The collision tells us only that a loop exists. To get its length, keep one pointer at the collision node and move around the cycle until it comes back to that same node.

8. Why the Counting Method Works

Suppose the collision happens at any node inside the loop. Starting there, following next repeatedly must eventually return to the same node because every node in the loop belongs to the cycle. The number of moves needed to return is exactly the number of nodes in the loop.

9. Edge Cases

- Empty list $\rightarrow 0$
- One node with next = null $\rightarrow 0$
- One node pointing to itself $\rightarrow 1$
- Linear list $\rightarrow 0$
- Loop containing every node $\rightarrow$ length of the whole list
- Loop beginning in the middle $\rightarrow$ count only the cyclic portion

10. Complexity Comparison

Hashing: $O(N)$ traversal time with map operations; $O(N)$ extra space.

Tortoise and Hare: $O(N)$ time and $O(1)$ extra space.

11. Quick Revision

Hashing:

1. Store node $\rightarrow$ first timer.
2. Traverse.
3. Repeated node found $\rightarrow$ current timer - stored timer.
4. No repetition $\rightarrow 0$.

Optimal:

1. slow = head, fast = head.
2. Move slow by 1, fast by 2.
3. Collision $\rightarrow$ loop exists.
4. Start count = 1.
5. Move around loop until returning to collision node.
6. Return count.
7. No collision $\rightarrow 0$.

12. Key Interview Point

The problem can be solved by first detecting the loop and then counting one complete cycle from the collision node. The optimal method avoids storing all nodes.

13. Note About the Provided Video

The provided video link could not be fetched directly here, so the uploaded transcript was used as the source for the explanation. The transcript explicitly covers the timer/map method, map complexity, the slow/fast collision, and the counter-based loop-length calculation. ■cite■turn0view0■